



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 10

Team No.: 9

Project Title: Denying Writes to Executables

Team Members

Roll Number	Student Name	Signature
2520090017	Tejasri	
2520090007	Dikshita	
2520090235	Rohitha	

1. Abstract

Executable files are critical components of an operating system, and unauthorized modification of these files can lead to program tampering, malware injection, and loss of system integrity. The project “Denying Writes to Executables” aims to develop a Linux-based mechanism that prevents unauthorized write operations on executable files while allowing legitimate execution and system operations. The proposed solution will identify executable files and control write access using Linux file permissions and system-level mechanisms. The project will demonstrate relevant Operating Systems and System Programming concepts such as file permissions, process and file management, system calls, and access control. Linux APIs and system calls can be used to monitor file operations and deny write requests made by unauthorized users or processes. The system will generate appropriate messages or logs when a write operation is blocked. The expected outcome is a simple security mechanism that preserves the integrity of executable files and reduces the risk of unauthorized modification. This project provides practical understanding of Linux file-system security and demonstrates how Operating Systems concepts can be applied to protect critical system resources

2. Problem Statement

Executable files contain programs that are essential for running applications and system services. If an unauthorized user or process is able to modify an executable, it may change the program's behavior, introduce malicious code, or compromise system security. In a normal Linux environment, file permissions provide access control, but improper permissions or compromised processes may still allow unwanted modifications. Therefore, there is a need for a mechanism that specifically prevents unauthorized write operations on executable files while preserving legitimate execution. This project focuses on implementing such a protection mechanism using Linux file permissions, system calls, and access-control concepts.

3. Objectives

1. To identify executable files in a Linux file system and determine their write permissions.
2. To prevent unauthorized users or processes from modifying protected executable files.
3. To demonstrate Linux file permissions, system calls, and access-control mechanisms for executable protection.
4. To record or report blocked write attempts and evaluate the effectiveness of the protection mechanism.

Proposed Methodology

The project will be implemented on a Linux/Ubuntu environment using C or C++. The system will first identify executable files based on their file properties and permissions. Protected executable files will be configured so that unauthorized users or processes cannot perform write operations on them.

The program will monitor or attempt file-access operations and check whether a requested operation involves writing to a protected executable. If the operation is unauthorized, the write request will be denied and an appropriate message or log will be generated. Legitimate execution of the protected executable will remain possible.

The implementation will demonstrate Linux file permissions and relevant system calls for opening, accessing, and modifying files. The project will be tested using different users and file-access scenarios to verify that unauthorized write operations are blocked while permitted operations continue to work.

Flow:

Identify executable → Check protection status → Detect write request → Verify permission → Allow legitimate access / Deny unauthorized write → Record the result.

Flow:

Initialize → Create Buffer → Initialize Semaphores → Start Producers & Consumers → Produce Data → Store in Buffer → Consume Data → Synchronize Access → Display Results → Terminate.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
File permissions	Control read and write

Access Control	Determine whether a user/process is authorized to modify a protected executable.
File Management	Identify, open, access, and manage executable files.
Process Management	Understand the process attempting to access or modify an executable.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090017	L.Tejasri	Linux file-permission implementation and executable identification
2520090007	S.Dikishita	C/C++ implementation of file-access control and system calls
2520090235	Rohitha	Testing different users, permissions, and unauthorized write scenarios

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system for developing, compiling, and executing the project.
C / C++	Main programming language used to implement the Producer-Consumer system.
GCC Compiler	Compiles the C source code into an executable program

8. Expected Outcome

The project is expected to provide a Linux-based mechanism that prevents unauthorized write operations on protected executable files. The system should allow authorized execution of the files while rejecting modification attempts from unauthorized users or processes. When a write operation is denied, the system should provide an appropriate error message and, where implemented, record the attempt in a log. The project will demonstrate practical application of Linux file permissions, access control, file management, and system calls. The final outcome will be a working prototype that improves executable-file integrity and provides hands-on understanding of Operating Systems and System Programming concepts.

9. GitHub Repository

GitHub Repository Link: <https://github.com/tejasrilankey/Operating-Systems-2520090017>

Repository Created: _____

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: Raghupathi M

Signature: Remarks:

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____